



Kernel Security: how2rootkit



Understand the load and unload flow. Unlink from every list the kernel uses to enumerate modules. Then "fail" to load a module that is actually running.

Agenda

The first half lays the groundwork: what a .ko actually is, how the kernel loads and tracks it. The second half is a back-and-forth between attacker and defender:

Setup: *what is a .ko, the insmod flow, where a module lives, the rmmod flow.*

Round 1. *Attacker: list_del_init. Defender: spots the /sys/module/ leak.*

Round 2. *Attacker: kobject_del to close the sysfs leak. Defender: walks mod_tree, which lives outside the list.*

Round 3. Attacker: skip `insmod` entirely. Build a carrier that "fails" to load while quietly installing a payload via `load_module()` directly. The defender of Round 2 still wins technically — but userland sees nothing happen.

Why Hide A Module

You are already loaded. Your code is resident at EL1. You have chardevs, kprobes, ftrace hooks, whatever you need.

What you do not want is a defender running:

```
lsmod
.... ghost
cat /proc/modules
ls /sys/module/
wall "shit shit someone loaded a rk"
rmmod ghost
```

Hiding a module is not about removing functionality. It is about cutting the strings that tie your struct module to the enumeration surfaces userland reads.

You are still in memory. You just want to be unseen.

What Is A .ko, Really?

A kernel module is an **ELF relocatable object**. Same file format as a `.o` from your compiler. The only thing that makes it "a module" is the `.modinfo` section and the presence of a `module_init/module_exit` pair.

```
$ readelf -h payload.ko
Type:                               REL (Relocatable file)
Machine:                            AArch64
$ readelf -S payload.ko | grep -E '.text|.data|.init|.modinfo|.rela'
.init.text      PROGBITS  0000000000000000
.exit.text      PROGBITS  0000000000000000
.rela.init.text REL      0000000000000000
.modinfo        PROGBITS  0000000000000000
.data           PROGBITS  0000000000000000
.bss           NOBITS   0000000000000000
```

Every section starts at address 0. That's what "relocatable" means — the linker has not assigned runtime addresses yet. That job belongs to whoever loads the file.

A .ko Is (mostly) A .so That Was Never Linked

- You already know GOT/PLT from the userland side. A .so is an ELF shared object with finalized addresses and a dynamic linker table. You `dlopen` it, `ld.so` fixes up the references, and you call into it.
- A .ko stops one stage earlier. The compiler produces section-per-function .o files; `kbuild` combines them into a single relocatable with unresolved external references to kernel symbols (`printk`, `kmalloc`, etc.). Nothing is fixed up at build time.

At `insmod`, **the kernel itself plays the linker**. It allocates runtime addresses, resolves externs from its own symbol table, applies relocation records, and jumps into `init_module`.

The loader lives in `kernel/module/main.c`. One function handles the whole thing: `load_module()`.

The insmod Flow

```
insmod foo.ko
|
▼
finit_module(fd, uargs, flags)           - syscall
|
▼
may_init_module() → capable(CAP_SYS_MODULE)
|
▼
init_module_from_file()
|— kernel_read_file(fd, ...)           - slurp .ko into kernel buffer
|
▼
load_module(&info, uargs, flags)         - the workhorse
|— module_sig_check()
|— elf_validity_cache_copy()           - parse ELF, cache sechdrs
|— layout_and_allocate()              - module_alloc() → text/data
|— apply_relocations()                - fix up every symbol ref
|— add_unformed_module()              - list_add_rcu to `modules`
|— mod_sysfs_setup()                  - /sys/module/<name>/
|— do_init_module(mod)                - run mod->init()
```

Every stage is in `kernel/module/main.c`. `load_module` is the single entry point that turns an ELF buffer into a running module.

Where A Module Lives In Memory

After `load_module` finishes, the module is scattered across three allocations:

1. **`struct module`** — one `kzalloc` from the kernel heap. Holds the name, list node, sysfs kobj, kallsyms pointer, init/exit pointers, refcount. This is the bookkeeping.
2. **`mod->mem[MOD_TEXT].base`** — `module_alloc()`'d `vmalloc` region in the AArch64 module area (`MODULES_VADDR..MODULES_END`, 2GB on arm64). Holds the actual compiled code. Executable. Indexed in a latched RB-tree called `mod_tree` so `__module_address(addr)` can find which module an instruction belongs to.
3. **`mod->mem[MOD_DATA].base`** — separate `module_alloc` for `.data`, `.bss`, `.rodata`. Non-executable.

Plus a few side allocations: per-module kallsyms, sysfs kobj, arg string. The point is: **the modules list does not own the memory**. It is just a name-keyed enumeration. You can unhook from the list without touching the memory.

The rmmmod Flow

```
rmmmod foo
|
▼
delete_module("foo", flags)           - syscall
|
▼
find_module_all(name, ..., true)      - walks `modules` list by name
|                                     returns NULL on miss
▼
mod->exit()                            - your __exit runs
|
▼
free_module(mod)
├─ module_param_sysfs_remove()
├─ mod_sysfs_tear_down()
├─ list_del_rcu(&mod->list)
├─ mod_tree_remove(mod)
├─ module_memfree(mod->mem[TEXT].base) - free the code
├─ module_memfree(mod->mem[DATA].base) - free the data
└─ kfree(mod)                        - free struct module
```

Three frees: text region, data region, struct module. The reverse of the load, minus ELF parsing. Everything keys off the initial `find_module_all` — if the name lookup fails, nothing downstream runs.

load / unload

Load does	Unload undoes
<code>module_alloc text + data</code>	<code>module_memfree both</code>
<code>list_add_rcu(&mod->list, &modules)</code>	<code>list_del_rcu(&mod->list)</code>
<code>mod_tree_insert(mod)</code>	<code>mod_tree_remove(mod)</code>
<code>mod_sysfs_setup</code>	<code>mod_sysfs_teardown</code>
<code>do_init_module → mod->init()</code>	<code>mod->exit()</code>

Hide/unhide

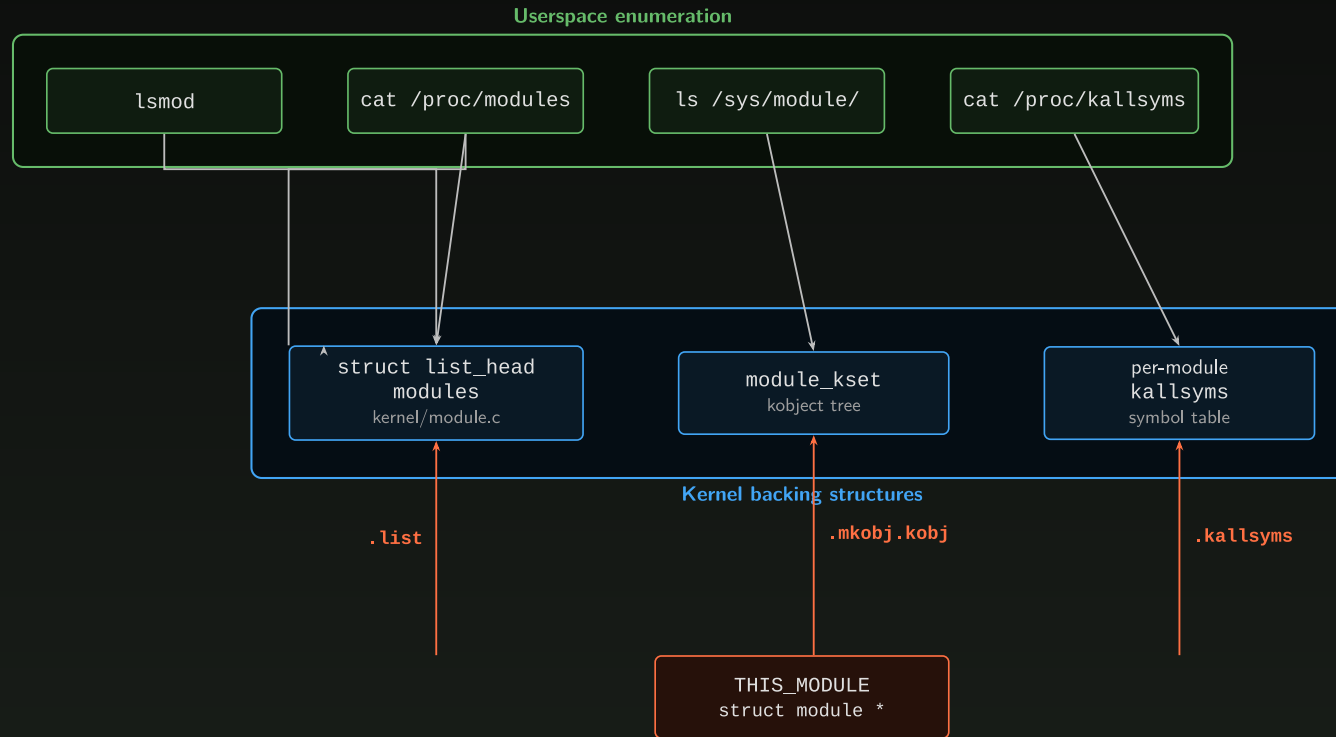
- Hiding a module is **breaking exactly one of these symmetries**.
- For ghostmod it is `list_del_init`
 - we do half of the unload, for the list only.
- The memory, the `mod_tree` entry, the `kobj`, the `kallsyms` table all stay.
- Safe unload is then just the inverse: re-thread the list and let `delete_module` run the full sequence.

Module enumeration

1. `/proc/modules` — walks the global modules list
2. `/sys/module/<name>/` — separate kobject hierarchy under `module_kset`
3. `/proc/kallsyms` — walks per-module symbol tables
4. `find_module(name)` — walks the modules list; used by `delete_module`

Hiding from one does not hide you from the others. Each is a separate unhook.

The Enumeration Surfaces



Three independent hooks. Hiding from one leaves the others visible.

struct module

```
struct module {
    enum module_state      state;
    struct list_head       list;           /* node in the global modules list */
    char                   name[MODULE_NAME_LEN];
    struct module_kobject  mkobj;         /* /sys/module/<name>/ */
    struct mod_kallsyms    *kallsyms;
    int                    (*init)(void);
    void                   (*exit)(void);
    atomic_t               refcnt;
    /* ... many more fields ... */
};
```

One of these is allocated for **every** loaded module. It is the kernel's bookkeeping for "you exist." Every enumeration surface reads a different field.

Working with modules

- `THIS_MODULE` is a per-module compile-time symbol.
- When you write `THIS_MODULE` in your code, the linker resolves it to `&__this_module`
 - a `struct module` instance that `kbuild` synthesizes into your `.ko` automatically. You never define it. It is your module's self-pointer.

struct-list-head recap: `list` is an intrusive node embedded **inside** `struct module`, not a container around it.

The modules List

```
/* kernel/module/main.c */  
LIST_HEAD(modules);  
DEFINE_MUTEX(module_mutex);
```

- Every loaded module is threaded through it via its embedded `list_head`.
- Adding a module calls `list_add_rcu(&mod->list, &modules)`.
- Removing calls `list_del_rcu`.
- `module_mutex` protects writers. Readers use RCU or grab the mutex.

/proc/modules (lsmod)

- lsmod basically cats /proc/modules and pretty-prints the output.
- /proc/modules is backed by a seq_file iterator that does one thing for each entry in modules:

```
for each `mod` in the global `modules`  
    list, print `mod->name`, size, refcount, dependencies, state, base address.
```

- The iterator macro is `list_for_each_entry(mod, &modules, list)`.
- If your `list_head` node is not threaded into `modules`, the iterator never visits you.
- The `seq_file` callback never runs on your behalf. `lsmod` won't print your name.

/sys/module

- When `load_module()` runs, it calls `mod_sysfs_setup()`, which creates a `kobject` under `module_kset` and registers it.
- That creates the `/sys/module/<name>/` directory.

```
/sys/module/ghostmod/  
  parameters/hidden  
  sections/  
  refcnt  
  initstate  
  holders/
```

This registration is independent of the `modules` list. Unlinking yourself from the list does not touch the `kobject`. `ls /sys/module/` still shows you.

find_module() and delete_module(2)

```
struct module *find_module(const char *name);
```

- Walks the modules list comparing mod->name against the argument.
- Returns the matching struct module * or NULL. Same list_for_each_entry pattern as every other consumer.

rmmod foo invokes the delete_module(2) syscall. The kernel side bottoms out in find_module_all("foo"), which is the same walk.

Hide From The List

```
list_del_init(&THIS_MODULE->list);
```

- One line of code removes you from the modules list.
- `list_for_each_entry` skips you. `find_module` returns NULL.
- `lsmod` prints nothing. `/proc/modules` is empty. `/proc/kallsyms` even loses your symbols (we'll see why).

Save `THIS_MODULE->list.prev` first so you can re-add yourself later, and make the toggle reachable from userland so you can demo it without reloading.

ghostmod: hide and unhide

A sysfs parameter callback toggles state. Two paths:

Hide:

1. Save `THIS_MODULE->list.prev` into a static variable
2. `list_del_init(&THIS_MODULE->list)`
3. Set `hidden = true`

Unhide:

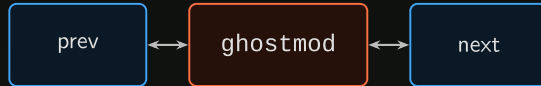
1. `list_add(&THIS_MODULE->list, saved_prev)` — puts the node back exactly where it was
2. Set `hidden = false`

Hide: save `list.prev`, unlink. The module stays in memory. Its `list_head` now points to itself.

Unhide: `list_add` after `saved_prev`. Back where you were.

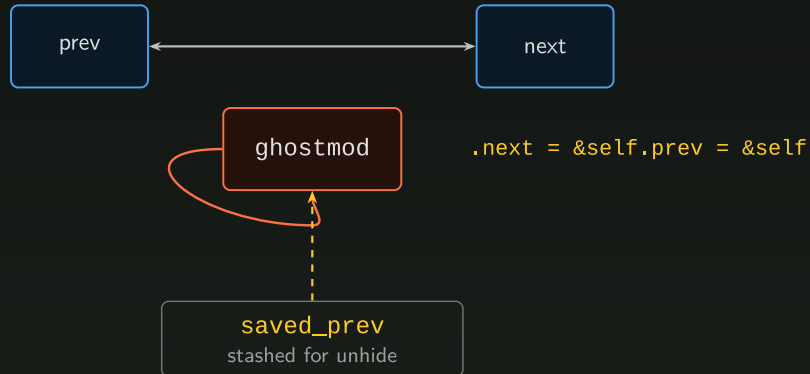
Hide/unhide

Before `list_del_init`



`lsmod` walks the chain, sees `ghostmod`.

After `list_del_init(&THIS_MODULE->list)`



Still in memory. Invisible to `lsmod`. Re-addable on demand.

list_del_init vs list_del

call	leaves entry->next/prev as	use it when
<code>list_del(entry)</code>	poison values (0x100, 0x122)	the node will never be touched again
<code>list_del_init(entry)</code>	self-pointers (entry, entry)	the node has a future — re-add, re-check

`list_del` is the standard "I'm done with this" call. Touching a poisoned node — even just calling `list_empty()` on it — will corrupt or crash whatever you splice it into.

`list_del_init` leaves the node in a clean, re-addable state.

`list_empty(entry)` is true. `list_add(entry, ...)` is safe. We need both properties: the unhide path will `list_add` later. So which one should ghostmod use?

list_del_init vs list_del

ghostmod uses list_del_init..

Sysfs Toggle

We need a way to flip `hidden` from userland without recompiling.
Cheapest option: a writable module parameter.

`module_param_cb(name, ops, var, mode)` registers a parameter with custom **callback ops**. The `ops` struct gives you a `.set` function — your code — that runs whenever userland writes to `/sys/module/<your-mod>/parameters/<name>`. Mode `0644` makes the file root-writable.

```
echo 1 > /sys/module/ghostmod/parameters/hidden # hide
echo 0 > /sys/module/ghostmod/parameters/hidden # unhide
```

How might you make this more "stealthy"?

What Hides, What Leaks

You unlinked from the modules list. Run the checks:

Surface	Hidden?	Why
lsmod	yes	reads /proc/modules
/proc/modules	yes	seq_file walks the modules list
find_module("ghostmod")	yes	walks the modules list
/proc/kallsyms (your symbols)	yes	module_get_kallsym walks the modules list
/sys/module/ghostmod/	no	separate module_kset kobject hierarchy
ftrace/kprobe registrations	no	their own tables
chardev in /dev	no	per-fs registrations

More detection surface

The leak is `/sys/module/<name>/`. That's not built from the list — it's a separate kobject tree. The next slides walk through that hole and what to do about it.

Anything you registered with `register_kprobe`, `ftrace_set_filter_ip`, `misc_register`, `proc_create` — all still visible on their respective tables. Each is its own list, each is its own unhook.

rmmod

Hide the module:

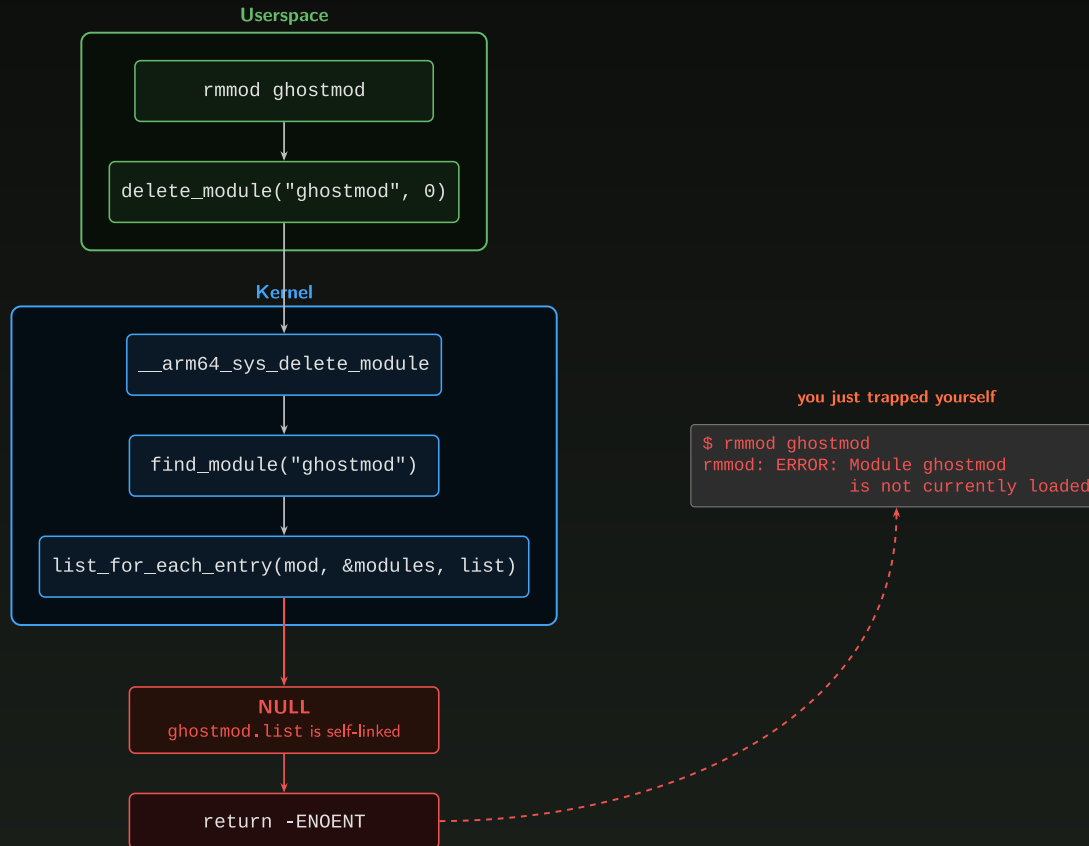
```
$ echo 1 > /sys/module/ghostmod/parameters/hidden  
$ lsmod | grep ghost  
(nothing)
```

Try to unload it:

```
$ sudo rmmod ghostmod  
rmmod: ERROR: Module ghostmod is not currently loaded
```

Why?

Why rmmod Fails



rmmod

The syscall in pseudocode:

```
delete_module(name, flags):
```

- 1. take module_mutex*
- 2. mod = find_module_all(name) (walks the modules list)*
- 3. if mod == NULL: return -ENOENT*
- 4. else: call mod->exit, free its pages*

You unlinked yourself from the modules list in step 2's input. Step 3 returns NULL. Userland prints "not currently loaded."

Your module is loaded. The kernel cannot find it by name. The kernel cannot unload it.

Just save it...

- Once you unlink from the modules list, you have no standard way back out.
- You are a permanent kernel resident until reboot.

Unless you plan ahead and just save the module struct. ...

ghostmod's Safety Net

```
echo 0 > /sys/module/ghostmod/parameters/hidden  
rmmod ghostmod
```

The unhide makes the module visible to `find_module_all` again. From that point on, `rmmod` is just a normal `rmmod`.

Safe Unload

1. `echo 0 > /sys/module/ghostmod/parameters/hidden`

(or chardev write, ioctl, magic signal, kprobe on `kill()`)

2. `set_hidden(val=0, kp)`

3. `list_add(&THIS_MODULE->list, saved_prev)`

4. `rmmod ghostmod` (now succeeds)

5. `__exit` runs, module unloads cleanly

The trick

The hidden param stays writable because we never deleted our own kobject.

Trigger channel $\neq$ enumeration channel.

Defender: The Sysfs Leak

- The attacker is hidden from `lsmod`, `/proc/modules`, `/proc/kallsyms`, and `find_module..`
- `/sys/module/<name>/` is built from a `kobject` registered separately during `load_module()`, under `module_kset`. **It does not walk the modules list.** It's its own enumeration.

Detection: /sys/module vs /proc/modules

```
comm -23 \  
<(ls /sys/module/ | sort) \  
<(awk '{print $1}' /proc/modules | sort)
```

- list every directory under /sys/module/, list every name in /proc/modules, print names in the first set but not the second. After `echo 1 > .../hidden`, this prints `ghostmod` because the `kobject` is still there but the list entry is gone.
- We could call `kobject_del(&THIS_MODULE->mkobj.kobj)` to remove the `sysfs` directory
 - that defeats this check, at the cost of losing your `parameters/hidden` trigger file.

Other Detection Signals

- `/sys/module/` directories with no `/proc/modules` entry
- ftrace filter lists containing IPs that kallsyms cannot attribute
- `register_kprobe` targets owned by no visible module
- Refcount drift: `init_mm`, `core_layout`, slab caches whose ownership traces to a ghost
- Memory scan: walk the kernel module region looking for `struct module` magic byte patterns, compare to the list walk

Hide From Sysfs Too

- The kobject under `module_kset` is its own data structure, registered separately by `mod_sysfs_setup()` during load.
- We can call the matching teardown function ourselves.
- If we hide from sysfs as well, the `comm` detector returns nothing
 - the visible-set on the left and the visible-set on the right are now equal.

The sysfs kobject

```
struct module_kobject {  
    struct kobject kobj;  
    struct module *mod;  
    struct kobject *drivers_dir;  
    struct module_param_attrs *mp;  
    struct completion *kobj_completion;  
};
```

`THIS_MODULE->mkobj.kobj` is the kobject backing `/sys/module/<name>/`. Remove it from sysfs with:

```
kobject_del(&THIS_MODULE->mkobj.kobj);
```

How do we talk to the Rootkit now?

If you `kobject_del` your own `mkobj.kobj`,
`/sys/module/ghostmod/parameters/hidden` disappears with the
directory.

Which means **your sysfs trigger is gone**. You have no way to flip `hidden`
back to 0 from userland. You are hidden forever. **hint** `man kill`

mod_tree

- It's an RB-tree keyed by address ranges, used internally by `__module_address(unsigned long addr)` to answer "which module owns this instruction pointer?"
- The unwinder calls it on every panic. kprobes call it on every probe registration.
- It is **independent of the modules linked list**.

`list_del_init` does not touch `mod_tree`. `kobject_del` does not touch `mod_tree`. Whatever the attacker did in Rounds 1 and 2, they are still in `mod_tree`.

Defender: walk the tree, compare to the visible list, the difference is a ghost.

ghosthunter_ex — The Homework Starter

- In `inclass/module_hiding/ghosthunter_ex/` you get a starter `.ko` that does step one of the real defense: walks the kernel modules linked list directly from kernel space and prints every entry.
- That alone is enough to beat **all** of the naive offense list above. Hook `lsmod` all you want — `ghosthunter_ex` does not call `lsmod`. Hide `/lib/modules/foo.ko` on disk all you want — the module is in memory, not on disk. The starter sees it.
- The homework: extend the starter so it **also** walks `mod_tree` and cross-references. That is the next slide. Three TODOs, ~30 minutes of work, and you have built a real ghost detector that the rest of this lecture's attackers cannot hide from.

Searching for Ghosts: mod_tree

The `modules` list is one enumeration of loaded modules. It is not the only one. `mod_tree` is an rbtrees keyed by address range, used internally for `__module_address(unsigned long addr)` — "which module owns this instruction pointer?"

`list_del_init` removes us from the list. `kobject_del` removes us from `sysfs`. Neither one touches the rbtrees.

```
addr in mod_tree AND owning module NOT in `modules` list == GHOST
```

Detection strategy: scan the module address region, call `__module_address` on each address, collect unique struct module pointers, subtract the visible list.

`__module_address` is not exported, so resolve it the same way `ghostloader` will resolve `load_module` — `kprobe` bootstrap.

ghosthunter.ko Scan Loop

The detector's main loop, in pseudocode:

```
for addr in MODULES_VADDR step PAGE_SIZE until  
MODULES_END: mod =  
__module_address(addr) (under preempt_disable)  
if mod and not yet seen: add to found[]
```

```
walk visible modules list → visible[]
```

```
for mod in found: if mod not in visible: that's a  
ghost
```

On arm64, MODULES_VADDR..MODULES_END is 2 GB of virtual address space. At PAGE_SIZE steps that is 512 K iterations. Most return NULL instantly via __module_address's internal `addr_min/addr_max` bounds check. The full scan finishes in milliseconds.

Ghost Found

```
$ insmod ghostmod.ko
$ echo 1 > /sys/module/ghostmod/parameters/hidden
$ insmod ghosthunter.ko
$ dmesg | grep ghosthunter
ghosthunter: starting ghost scan
ghosthunter: __module_address @ fffff80008015fdd0
ghosthunter: visible modules via list walk: 2
ghosthunter: unique modules via mod_tree scan: 2
ghosthunter: GHOST found - struct module *ffff80007b0d2040 name='ghostmod'
ghosthunter: 1 ghost module(s) detected
```

ghosthunter sees itself and ghostmod via `mod_tree`. The visible list walk only sees ghosthunter (ghostmod unlinked itself). Set difference yields one ghost by name.

The lesson: **hiding from the list is not enough**. Serious detection reads the tree, not the list. Any defender with kernel-level visibility can find you in ~50 lines of code. The attacker needs an entirely different approach.

Don't Insmod Normally At All

Plan:

1. Build a "carrier" module that ships an embedded payload .ko in its data section
2. The carrier's init function calls `load_module()` **directly** to install the payload
3. Then the carrier's init **returns an error**, so the kernel tears down the carrier
4. Userland sees `insmod: ERROR: ... Invalid parameters` and shrugs
5. The payload, however, was loaded by `load_module()` and is alive — it self-hides immediately

Two Modules

```
inc/lass/module_hiding/loader_demo/  
├── payload/  
│   └── payload.c  
├── ghostloader.c  
└── payload_blob.h
```

← kprobe on __arm64_sys_kill
list_del_init self in init
← #include "payload_blob.h"
vmalloc + memcpy payload bytes
load_module() → payload_init()
return -EINVAL
← xxd -i payload.ko (generated)

The build chains payload → xxd -i → blob header → ghostloader. At runtime ghostloader does not touch the filesystem. The payload bytes ship inside ghostloader's .data.

Same idea as memfd reflective loading on the userland side: embed code, materialize it, execute, dispose of the carrier.

Resolving load_module

`load_module` is static in `kernel/module/main.c`. Not exported. Not visible to any in-tree symbol resolver.

But with `CONFIG_KALLSYMS_ALL=y` it still appears in `/proc/kallsyms`, and **any function in kallsyms can be resolved by the kprobe bootstrap** you already used for `sys_call_table`:

Build a struct `kprobe` with `.symbol_name = "load_module"`. `register_kprobe()` succeeds and fills in `kp.addr` with the resolved address. `unregister_kprobe()`. Keep the address.

You now have a function pointer to a static kernel function the kernel never intended to share.

Building A struct load_info By Hand

```
struct gl_load_info {
    const char *name;
    struct module *mod;
    Elf_Ehdr *hdr;           /* ← our buffer */
    unsigned long len;       /* ← payload size */
    Elf_Shdr *sechdrs;       /* zeroed; load_module fills in */
    char *secstrings, *strtab; /* zeroed */
    unsigned long symoffs, stroffs, init_typeoffs, core_typeoffs;
    bool sig_ok;
    unsigned long mod_kallsyms_init_off;
    struct { unsigned int sym, str, mod, vers, info, pcpu; } index;
};
```

The struct is static in `kernel/module/internal.h`, so modules cannot include it. Copy the layout verbatim. Zero everything, set `.hdr = buf`, `.len = payload_ko_len`. `load_module` fills in the rest as it parses the ELF.

Match the kernel's `.config`: `KALLSYMS=y`, `MODULE_DECOMPRESS=n`, `MODULE_STATS=n`. A mismatch silently corrupts adjacent fields.

The uargs Trick: intentional error

Near the end, `load_module` does:

```
mod->args = strndup_user(uargs, ~0UL >> 1);
```

`strndup_user(NULL, ...)` returns `-EFAULT`. We need a **valid userspace pointer to a null byte**. From kernel context. Without a userland process of our own.

```
user_uargs = vm_mmap(NULL, 0, PAGE_SIZE,  
    PROT_READ | PROT_WRITE,  
    MAP_PRIVATE | MAP_ANONYMOUS, 0);
```

`current->mm` is still valid — we are running inside `finit_module`, called by the `insmod` process that is blocked in the syscall. Anonymous pages are zero-filled on first access, so byte 0 already holds `'\0'`. `strndup_user` reads one byte, returns a `kmalloc`'d empty string, `load_module` moves on.

`vm_munmap` it after the call. © 2026 Chonky LTD

The error

The carrier's `init` ends with these steps:

1. Call `resolved load_module(&info, user_uargs, 0)` — payload loads, payload's `init` runs synchronously, payload self-hides
2. `vm_munmap` the fake user page
3. Return `-EINVAL`

The payload is in a **completely separate allocation**, owned by its own `struct module`. Freeing ghostloader does not touch one byte of payload's memory. The kprobe registered by `payload_init()` points into payload's text region, which is still live.

User's terminal:

```
insmod: ERROR: could not insert module ghostloader.ko: Invalid parameters
```

kill -62 1337

`payload.c` registered a kprobe on `__arm64_sys_kill` during its init. Every `kill()` syscall hits our handler.

The handler's job, in pseudocode:

```
read pid and sig from the user's pt_regs if pid !=  
1337 or sig != 62: do nothing if currently hidden:  
list_add self back into modules list
```

magic kill

- foo bar `__arm64_sys_kill` is the AArch64 syscall wrapper. The wrapper ABI passes a single `const struct pt_regs *` in `x0`, and the user's syscall arguments live as `x0` and `x1` inside that struct. So `regs->regs[0]->regs[0]` is the user's `pid` and `regs->regs[0]->regs[1]` is the user's `sig`.

`list_add` is pure pointer manipulation — no allocation, no sleeping — so it is safe to call from kprobe handler context (which is atomic).

Once the handler returns and the kprobe finishes, the module is back on the list. `rmmod payload` is now a normal unload.

Lab Demo

```
$ insmod /mnt/shared/module_hiding/loader_demo/ghostloader.ko
insmod: ERROR: could not insert module ghostloader.ko: Invalid parameters

$ lsmod | grep -E 'payload|ghostloader'
(nothing)

$ dmesg | tail -5
ghostloader: init running - about to load 48576-byte payload
ghostloader: load_module @ ffff800008015eac8
payload: loaded (will self-hide immediately)
payload: hidden from lsmod (list_empty=1)
ghostloader: payload loaded, self-destructing

$ kill -62 1337

$ lsmod | grep payload
payload                16384  0

$ rmmod payload
```


Arms Race: Beyond ghosthunter

ghosthunter's signal is "in mod_tree, not in list." What would it take to beat it?

1. **Also hide from mod_tree** — the tree rebalance logic is not exported, but `mod_tree_remove` is a kallsyms symbol. Kprobe bootstrap, call directly, accept the race.
2. **Loader lives in raw vmalloc** — no `module_alloc`, never enters `mod_tree`. Then you hand-manage relocations and execute permissions. This is what `exploit_kload_hard_solution.c` does.
3. **Stage all code in .text of a benign module** — the benign module stays visible, you hide only the attack state. Harder to attribute.

Each layer the attacker climbs forces the defender up one too. Defenders eventually reach: "cross-check every kernel code pointer against a whitelist." At that point it stops being module hiding and becomes code provenance.

